

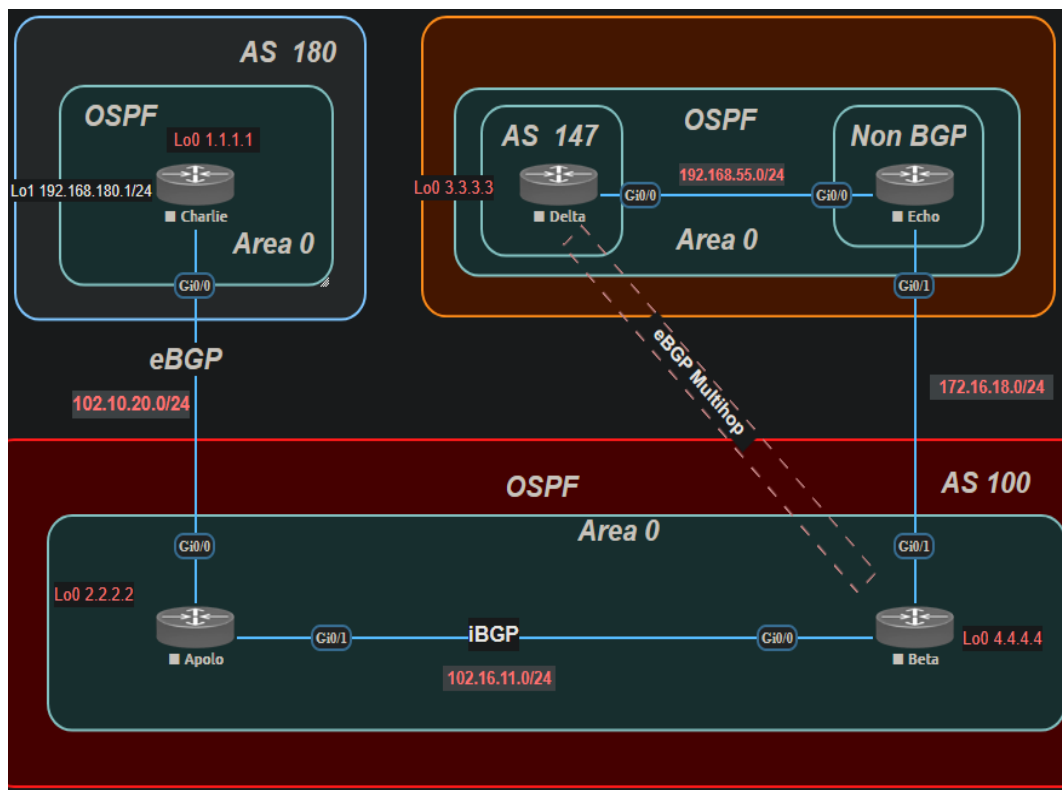
BGP

B O R D E R G A T E W A Y P R O T O C O L

A D V A N C E D L A B M A N U A L — V O L U M E 2

Route Summarization • Filtering • Regex • BGP Communities

Distribute Lists | Prefix Lists | ACLs | Route Maps | Route Tagging



1. Introduction & Topology Recap

This lab continues from Volume 1. All BGP sessions must be in Established state before applying any configuration from this volume. The topology is unchanged — review it carefully before starting, paying attention to which router peers with which AS.

WARN Complete BGP Lab Manual Vol. 1 before starting. All BGP sessions must be Established and prefixes exchanging. OSPF (IPv4) and OSPFv3 (IPv6) must be fully converged.

1.1 Topology Reference

AS	Router	Interfaces	BGP Peers	Role
AS 100	Apolo	Gi0/0 (→Charlie), Gi0/1 (→Beta)	Charlie (eBGP, AS180), Beta (iBGP, AS100)	Ingress eBGP / iBGP speaker
AS 100	Beta	Gi0/0 (→Apolo), Gi0/1 (→Echo)	Delta (eBGP multihop, AS147), Apolo (iBGP, AS100)	Egress eBGP / iBGP speaker
AS 147	Delta	Gi0/0 (→Echo), Lo0	Beta (eBGP multihop, AS100)	Transit peer
AS 147	Echo	Gi0/0 (→Delta), Gi0/1 (→Beta)	None — OSPF transit only	Non-BGP multihop relay
AS 180	Charlie	Gi0/0 (→Apolo), Lo1	Apolo (eBGP, AS100)	Single-homed customer

KEY Apolo peers only with Charlie (eBGP) and Beta (iBGP). Apolo has NO direct BGP session with Delta. Delta's multihop eBGP session is with Beta (Lo0 3.3.3.3 ↔ Lo0 4.4.4.4), travelling through Echo as transit.

1.2 BGP Session Summary

Session	Peers	Type	Physical Path
Charlie Gi0/0 ↔ Apolo Gi0/0	AS180 ↔ AS100	eBGP direct	102.10.20.0/24 link
Delta Lo0 ↔ Beta Lo0	AS147 ↔ AS100	eBGP Multihop (2 hops)	Delta→Echo Gi0/0→Echo Gi0/1→Beta
Apolo Gi0/1 ↔ Beta Gi0/0	AS100 ↔ AS100	iBGP	102.16.11.0/24 link
Beta Gi0/1 ↔ Echo Gi0/1	AS100 ↔ AS147	No BGP	Transit for Delta↔Beta TCP
Echo Gi0/0 ↔ Delta Gi0/0	AS147 internal	No BGP	OSPF within AS147

1.3 Prefix Inventory for This Lab

These prefixes are used across summarization, filtering, and community exercises. Add loopback interfaces as shown before injecting into BGP.

Router	Prefix(es)	Source	Lab Purpose
Charlie	192.168.180.0/24	Lo1 (existing)	Base customer route
Charlie	10.180.1.0/24, 10.180.2.0/24, 10.180.3.0/24	Lo2, Lo3, Lo4	Summarisation components → 10.180.0.0/22
Apolo	10.100.0.0/24	Lo1 (existing)	AS100 block component
Beta	10.100.1.0/24	Lo1 (existing)	AS100 block component → aggregate 10.100.0.0/22
Delta	172.20.0.0/24	Lo1 (existing)	AS147 base prefix
Delta	172.20.1.0/24, 172.20.2.0/24	Lo2, Lo3	Filtering exercise — more-specifics

2. Route Summarization

2.1 Concepts

BGP summarization reduces routing table size, hides internal topology, and improves convergence. The aggregate-address command has several options:

- **summary-only** — suppresses ALL component routes; only the aggregate is advertised.
- **as-set** — includes AS_PATH of all components in the aggregate (preserves loop prevention).
- **suppress-map** — suppresses only components matched by a route-map; others still advertised.
- **attribute-map** — applies attributes (e.g., community) to the aggregate itself.

NOTE

The aggregate only appears in the BGP table if at least one component route exists in the BGP table. Always inject component routes via 'network' statements before configuring aggregate-address.

2.2 Task 1 – Summarize Charlie's Customer Routes (AS 180)

Charlie has Lo1 (192.168.180.0/24) plus three new loopbacks: Lo2=10.180.1.0/24, Lo3=10.180.2.0/24, Lo4=10.180.3.0/24. The goal is to advertise a single summary 10.180.0.0/22 to Apolo and suppress the component /24s.

Step 1 – Add loopbacks and inject components into BGP

```
! — Charlie (AS 180) —————
Charlie(config)# interface loopback 2
```

```

Charlie(config-if)# ip address 10.180.1.1 255.255.255.0
Charlie(config)# interface loopback 3
Charlie(config-if)# ip address 10.180.2.1 255.255.255.0
Charlie(config)# interface loopback 4
Charlie(config-if)# ip address 10.180.3.1 255.255.255.0

Charlie(config)# router bgp 180
Charlie(config-router)# address-family ipv4 unicast
Charlie(config-router-af)# network 10.180.1.0 mask 255.255.255.0
Charlie(config-router-af)# network 10.180.2.0 mask 255.255.255.0
Charlie(config-router-af)# network 10.180.3.0 mask 255.255.255.0
Charlie(config-router-af)# exit-address-family

```

Step 2 – Configure aggregate-address with summary-only

```

! — Charlie (AS 180) —————
Charlie(config)# router bgp 180
Charlie(config-router)# address-family ipv4 unicast
Charlie(config-router-af)# aggregate-address 10.180.0.0 255.255.252.0 summary-
only
Charlie(config-router-af)# exit-address-family

! Verify locally — components should show 's' (suppressed)
Charlie# show bgp ipv4 unicast
! 'a>' on 10.180.0.0/22 = aggregated best path
! 's' on 10.180.1-3.0/24 = suppressed

! Verify on Apollo — only aggregate should arrive
Apollo# show bgp ipv4 unicast neighbors 102.10.20.1 received-routes
! Only 10.180.0.0/22 should appear — no /24 components

! Verify the aggregate propagates to Beta via iBGP
Beta# show bgp ipv4 unicast 10.180.0.0/22
! Next-hop should be 102.16.11.1 (Apollo, next-hop-self)

```

NOTE

The aggregate is NOT sent by Apollo — it is received FROM Charlie. Apollo propagates it to Beta via iBGP with next-hop-self. Filtering exercises in Section 3 will use Apollo as the policy enforcement point for routes received from Charlie.

2.3 Task 2 – Aggregate AS 100 Block (Apollo + Beta)

Apollo has Lo1=10.100.0.0/24 and Beta has Lo1=10.100.1.0/24. Together these form a block that can be summarised as 10.100.0.0/22 (covers .0 through .3 — leaving room for future growth). Apollo advertises the aggregate outbound to Charlie. The summary is generated on Apollo since it has the full picture via iBGP from Beta.

Step 1 – Verify both components are in Apollo's BGP table

```

! Apollo's own Lo1 is injected via 'network' (from Vol.1 config)
! Beta's Lo1 arrives via iBGP from Beta
Apollo# show bgp ipv4 unicast | include 10.100

```

```
! Expected:
! *> 10.100.0.0/24 0.0.0.0 0 32768 i (Apolo local)
! *>i 10.100.1.0/24 102.16.11.2 0 100 i (Beta via iBGP)
```

Step 2 – Configure aggregate with as-set on Apolo

```
! — Apolo (AS 100) —————
Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# aggregate-address 10.100.0.0 255.255.252.0 summary-only
as-set
Apolo(config-router-af)# exit-address-family

! Verify aggregate appears with ATOMIC_AGGREGATE and AGGREGATOR
Apolo# show bgp ipv4 unicast 10.100.0.0/22
! Look for: 'atomic-aggregate' and 'aggregated by 100 2.2.2.2'

! Verify it is advertised to Charlie (eBGP)
Apolo# show bgp ipv4 unicast neighbors 102.10.20.1 advertised-routes | include
10.100
! Only 10.100.0.0/22 should appear — not the /24 components

! Charlie should receive the aggregate
Charlie# show bgp ipv4 unicast 10.100.0.0/22
```

KEY

as-set is important here because Beta's Lo1 (10.100.1.0/24) originated in AS 100 and was advertised by Beta. Without as-set the aggregate has an empty AS_PATH, which could cause routing loops if Charlie re-advertised it back. as-set preserves the AS_PATH of all component routes.

2.4 Task 3 – Selective Suppression with suppress-map

Scenario: Apolo advertises 10.100.0.0/22 to Charlie but should still advertise 10.100.0.0/24 specifically to Delta via Beta's eBGP multihop session (for traffic engineering). Use suppress-map to suppress only Apolo's own /24 outbound, while leaving Beta's /24 (10.100.1.0/24) unsuppressed toward Delta.

KEY

The suppress-map route-map uses match statements only — set statements are ignored. Any prefix matched by the route-map IS suppressed; unmatched prefixes continue to be advertised.

```
! — Apolo (AS 100) —————
! Define which component to suppress (Apolo's own /24 only)
Apolo(config)# ip prefix-list SUPPRESS-APOLO-ONLY seq 5 permit 10.100.0.0/24

Apolo(config)# route-map SUPPRESS-MAP permit 10
Apolo(config-route-map)# match ip address prefix-list SUPPRESS-APOLO-ONLY
Apolo(config-route-map)# exit

! Replace summary-only aggregate with suppress-map aggregate
Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
```

```

Apolo(config-router-af)# no aggregate-address 10.100.0.0 255.255.252.0 summary-
only as-set
Apolo(config-router-af)# aggregate-address 10.100.0.0 255.255.252.0 as-set
suppress-map SUPPRESS-MAP
Apolo(config-router-af)# exit-address-family

! Result:
! 10.100.0.0/24 - suppressed (matched by SUPPRESS-MAP)
! 10.100.1.0/24 - still advertised (not matched, passes through Beta→Delta)
! 10.100.0.0/22 - aggregate advertised to all peers

Apolo# show bgp ipv4 unicast | include 10.100
! 'a>' on /22, 's' on 10.100.0.0/24, '*>i' on 10.100.1.0/24

```

3. Route Filtering

3.1 Filtering Tools Comparison

Tool	Matches On	Mask Precision	Can Set Attributes?
distribute-list (std ACL)	Network address only	None — no mask control	No
distribute-list (ext ACL)	Network + mask (as dst)	Yes — wildcard mask	No
prefix-list (neighbor filter)	Prefix + mask range (le/ge)	High	No
route-map	Prefix, AS-path, community, metric	Very High	Yes
filter-list (AS-path ACL)	AS_PATH attribute via regex	High (regex)	No (use route-map)

3.2 Task 4 – Distribute-List with Standard ACL

Scenario: Apolo receives 10.180.1.0/24, 10.180.2.0/24, and 10.180.3.0/24 from Charlie (when summary-only is not active). Apolo should not forward 10.180.3.0/24 to Beta via iBGP. Apply a distribute-list with a standard ACL outbound on the iBGP session toward Beta.

```

! — Apolo (AS 100) —————
Apolo(config)# ip access-list standard BLOCK-10-180-3
Apolo(config-std-nacl)# deny 10.180.3.0 0.0.0.255
Apolo(config-std-nacl)# permit any
Apolo(config-std-nacl)# exit

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.16.11.2 distribute-list BLOCK-10-180-3 out
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.16.11.2 soft out

! Verify: 10.180.3.0/24 absent from Beta's BGP table
Beta# show bgp ipv4 unicast | include 10.180.3

```

```
! Expected: no output
Apolo# show bgp ipv4 unicast neighbors 102.16.11.2 advertised-routes | include 10.180
```

WARN

Standard ACLs match only the network address — not the mask length. 'deny 10.180.3.0 0.0.0.255' blocks any prefix whose address falls in that range regardless of its mask. Use extended ACLs or prefix-lists when mask-length precision is needed.

3.3 Task 5 – Distribute-List with Extended ACL (Mask Matching)

Scenario: Apollo receives Delta's more-specific routes (172.20.1.0/24 and 172.20.2.0/24) via iBGP from Beta. Apollo should only accept Delta prefixes that are /24 or shorter (i.e., /16 through /24). Block anything more specific than /24 arriving from Beta.

KEY

Extended ACL route filtering: Source = network address with wildcard. Destination = subnet mask in inverse (wildcard). '255.255.255.0 0.0.0.0' matches exactly /24. '255.255.0.0 0.0.255.255' matches /16 through /32 depending on wildcard bits.

```
! — Apollo (AS 100) —————
! Permit 172.20.x.x with mask exactly /24 or shorter (permit /16 to /24)
Apolo(config)# ip access-list extended PERMIT-172-TO-24
Apolo(config-ext-nacl)# remark Permit 172.20.0.0/16 summary
Apolo(config-ext-nacl)# permit ip 172.20.0.0 0.0.255.255 255.255.0.0 0.0.0.0
Apolo(config-ext-nacl)# remark Permit up to /24
Apolo(config-ext-nacl)# permit ip 172.20.0.0 0.0.255.255 255.255.255.0 0.0.0.0
Apolo(config-ext-nacl)# remark Deny anything more specific
Apolo(config-ext-nacl)# deny ip 172.20.0.0 0.0.255.255 any
Apolo(config-ext-nacl)# permit ip any any
Apolo(config-ext-nacl)# exit

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.16.11.2 distribute-list PERMIT-172-TO-24 in
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.16.11.2 soft in

Apolo# show bgp ipv4 unicast | include 172.20
! 172.20.0.0/24 and /16 should appear; nothing longer than /24
```

3.4 Task 6 – Prefix-List Filtering

Scenario A – Accept only 10.180.0.0/16 subnets between /22 and /24 from Charlie

```
! — Apollo (AS 100) —————
Apolo(config)# ip prefix-list FROM-CHARLIE seq 5 permit 10.180.0.0/16 ge 22 le 24
Apolo(config)# ip prefix-list FROM-CHARLIE seq 10 deny 0.0.0.0/0 le 32

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
```

```

Apolo(config-router-af)# neighbor 102.10.20.1 prefix-list FROM-CHARLIE in
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.10.20.1 soft in

Apolo# show ip prefix-list FROM-CHARLIE
Apolo# show bgp ipv4 unicast neighbors 102.10.20.1 received-routes
! Only 10.180.x.x/22-24 prefixes should be accepted

```

Scenario B – Block default route advertisement outbound on all eBGP sessions

A safety policy ensuring AS 100 never accidentally advertises a default route to any external peer. Applied outbound on both eBGP sessions — Apollo toward Charlie, and Beta toward Delta.

```

! — Apollo (AS 100) — outbound toward Charlie —————
Apolo(config)# ip prefix-list NO-DEFAULT seq 5 deny 0.0.0.0/0
Apolo(config)# ip prefix-list NO-DEFAULT seq 10 permit 0.0.0.0/0 le 32

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.10.20.1 prefix-list NO-DEFAULT out
Apolo(config-router-af)# exit-address-family

! — Beta (AS 100) — outbound toward Delta (multihop) —————
Beta(config)# ip prefix-list NO-DEFAULT seq 5 deny 0.0.0.0/0
Beta(config)# ip prefix-list NO-DEFAULT seq 10 permit 0.0.0.0/0 le 32

Beta(config)# router bgp 100
Beta(config-router)# address-family ipv4 unicast
Beta(config-router-af)# neighbor 3.3.3.3 prefix-list NO-DEFAULT out
Beta(config-router-af)# exit-address-family

```

4. AS-Path Filtering with Regular Expressions

4.1 Regex Reference Table

Symbol	Meaning	BGP Example
^	Start of AS_PATH	^\$ = locally originated (empty path)
\$	End of AS_PATH	^180\$ = originated in AS180 only
_	Space/delimiter/ boundary	_180_ = 180 anywhere as a whole ASN
.	Any single character	^.* = any non-empty path
*	Zero or more of preceding	_180* = 180 repeated zero or more times
+	One or more of preceding	[0-9]+ = one or more digits (any ASN)
[]	Character class	[0-9] any digit; [^1] not-1
()	Grouping / alternation	^(180 147)\$ = from AS180 OR AS147

Symbol	Meaning	BGP Example
	OR operator	<code>^180\$ ^147\$</code> same as above

KEY

The underscore '_' is special in IOS: it matches a space, comma, left/right brace, or the start/end of the string. This makes '_147_' match AS147 appearing anywhere in a multi-ASN path — flanked by other ASNs or at the boundary.

4.2 Task 7 – filter-list: Block Routes Transiting AS 147 (Apolo)

Scenario: Apolo should not accept any prefix from Charlie whose AS_PATH shows it passed through AS 147 before reaching AS 100. Charlie could theoretically re-advertise AS147 routes back to AS100 — this filter prevents that.

```
! — Apolo (AS 100) —————
! Deny any path containing 147 as an ASN element
Apolo(config)# ip as-path access-list 10 deny _147_
Apolo(config)# ip as-path access-list 10 permit .*

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.10.20.1 filter-list 10 in
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.10.20.1 soft in

! Test regex interactively before applying
Apolo# show bgp ipv4 unicast regexp _147_
! Shows any routes currently in BGP table with 147 in AS_PATH
Apolo# show ip as-path-access-list 10
```

4.3 Task 8 – filter-list: Accept Only Single-Hop Paths (Beta)

Scenario: Beta should only accept prefixes originated directly in the AS it is peering with (AS_PATH of exactly one ASN — Delta, AS 147). This prevents Beta from accepting routes that passed through additional ASes before Delta.

```
! — Beta (AS 100) —————
! ^[0-9]+$ = start, one-or-more digits, end = exactly one ASN
Beta(config)# ip as-path access-list 20 permit ^[0-9]+$
Beta(config)# ip as-path access-list 20 deny .*

Beta(config)# router bgp 100
Beta(config-router)# address-family ipv4 unicast
Beta(config-router-af)# neighbor 3.3.3.3 filter-list 20 in
Beta(config-router-af)# exit-address-family
Beta# clear ip bgp 3.3.3.3 soft in

Beta# show bgp ipv4 unicast regexp ^[0-9]+$
! Should show only routes with a single-ASN path from Delta
```

4.4 Task 9 – filter-list: Permit Only Trusted ASes (Apolo)

Scenario: Apolo should accept eBGP routes only from prefixes originated in AS 180 or AS 147. Use alternation to match either AS in a single filter-list.

```
! — Apolo (AS 100) —————
! ^(180|147)$ = path consisting of ONLY AS180 OR ONLY AS147
Apolo(config)# ip as-path access-list 30 permit ^(180|147)$
Apolo(config)# ip as-path access-list 30 deny .*

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.10.20.1 filter-list 30 in
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.10.20.1 soft in

! Verify only AS180 routes accepted from Charlie
Apolo# show bgp ipv4 unicast regexp ^(180|147)$
```

4.5 Task 10 – Regex in route-map (match as-path + set attribute)

A route-map can use an AS-path ACL in a match clause and simultaneously set attributes. Here Apolo matches routes from AS 180 and sets LOCAL_PREF=200 so all AS 100 routers prefer exiting toward Charlie for AS 180 prefixes.

```
! — Apolo (AS 100) —————
Apolo(config)# ip as-path access-list 40 permit ^180$

Apolo(config)# route-map FROM-AS180-POLICY permit 10
Apolo(config-route-map)# match as-path 40
Apolo(config-route-map)# set local-preference 200
Apolo(config-route-map)# exit
Apolo(config)# route-map FROM-AS180-POLICY permit 20
Apolo(config-route-map)# exit

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.10.20.1 route-map FROM-AS180-POLICY in
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.10.20.1 soft in

! Verify LP on AS180 routes
Apolo# show bgp ipv4 unicast 192.168.180.0/24
! localpref 200 should appear

! Verify LP propagated to Beta via iBGP
Beta# show bgp ipv4 unicast 192.168.180.0/24
! localpref 200 should appear on Beta as well
```

5. Advanced Route-Map Techniques

5.1 Route-Map Structure Reference

```
route-map NAME permit|deny SEQUENCE
  match <condition>
  set <attribute>

! Applied to BGP neighbor — always inside address-family on IOS-XE:
router bgp ASN
  address-family ipv4 unicast
    neighbor X route-map NAME in|out

! Match options:
!   match ip address {prefix-list NAME | access-list NAME}
!   match as-path ACL-NUMBER
!   match community COMMUNITY-LIST-NAME
!   match metric VALUE

! Set options:
!   set local-preference VALUE
!   set weight VALUE
!   set metric VALUE
!   set as-path prepend ASN [ASN ...]
!   set community VALUE [additive]
!   set ip next-hop ADDRESS
```

KEY

On IOS-XE, 'neighbor X route-map NAME in|out' must be configured inside 'address-family ipv4 unicast' (or ipv6 unicast). Placing it under 'router bgp' without an address-family context is rejected or silently ignored on IOS-XE.

5.2 Task 11 – Multi-Clause Route-Map: Apollo Inbound from Charlie

Scenario: Apollo applies a three-clause inbound policy on routes from Charlie: (1) 10.180.1.0/24 gets weight 300, (2) 10.180.2.0/24 gets weight 200 and LP 150, (3) all other routes get default weight 100. Each clause is a separate sequence number.

```
! — Apollo (AS 100) —————
Apollo(config)# ip prefix-list PL-180-1 seq 5 permit 10.180.1.0/24
Apollo(config)# ip prefix-list PL-180-2 seq 5 permit 10.180.2.0/24

! Clause 10 — 10.180.1.0/24 → weight 300
Apollo(config)# route-map CHARLIE-INBOUND permit 10
Apollo(config-route-map)# match ip address prefix-list PL-180-1
Apollo(config-route-map)# set weight 300
Apollo(config-route-map)# exit

! Clause 20 — 10.180.2.0/24 → weight 200, LP 150
Apollo(config)# route-map CHARLIE-INBOUND permit 20
Apollo(config-route-map)# match ip address prefix-list PL-180-2
Apollo(config-route-map)# set weight 200
Apollo(config-route-map)# set local-preference 150
Apollo(config-route-map)# exit
```

```
! Clause 30 - catch-all, weight 100
Apolo(config)# route-map CHARLIE-INBOUND permit 30
Apolo(config-route-map)# set weight 100
Apolo(config-route-map)# exit

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.10.20.1 route-map CHARLIE-INBOUND in
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.10.20.1 soft in

Apolo# show bgp ipv4 unicast 10.180.1.0/24
! weight: 300
Apolo# show bgp ipv4 unicast 10.180.2.0/24
! weight: 200, localpref: 150
```

5.3 Task 12 – Outbound next-hop Override (Beta toward Delta)

Scenario: Beta must advertise prefixes to Delta with a specific next-hop (Beta's Lo1 address 10.100.1.1) instead of its Lo0 (4.4.4.4). Use 'set ip next-hop' in an outbound route-map on Beta's eBGP multihop session toward Delta.

```
! — Beta (AS 100) —————
Beta(config)# route-map SET-NH-LO1 permit 10
Beta(config-route-map)# set ip next-hop 10.100.1.1
Beta(config-route-map)# exit

Beta(config)# router bgp 100
Beta(config-router)# address-family ipv4 unicast
Beta(config-router-af)# neighbor 3.3.3.3 route-map SET-NH-LO1 out
Beta(config-router-af)# exit-address-family
Beta# clear ip bgp 3.3.3.3 soft out

! Verify on Delta: next-hop should be 10.100.1.1 not 4.4.4.4
Delta# show bgp ipv4 unicast 10.100.0.0/22
! Next-hop: 10.100.1.1
```

6. BGP Communities & Route Tagging

6.1 Community Concepts

A BGP community is a 32-bit tag carried with a prefix through UPDATE messages. Communities allow routers to apply consistent policy to groups of routes without enumerating individual prefixes.

Community	Value	Meaning
internet	0	Advertise to all BGP peers (default)
no-export	0xFFFFFFFF01	Do not advertise beyond this AS
no-advertise	0xFFFFFFFF02	Do not advertise to any peer
local-AS	0xFFFFFFFF03	Do not send outside local confederation
Custom (ASN:NN)	e.g., 100:10, 100:180	Private policy tagging

WARN

Communities must be explicitly sent using 'neighbor X send-community'. Without this, communities are stripped from outgoing updates silently — even if set by a route-map. Always pair 'set community' with 'neighbor X send-community both'.

6.2 Enable New-Format Communities

```
! Enable AA:NN notation on ALL routers participating in community exchange
Charlie(config)# ip bgp-community new-format
Apolo(config)# ip bgp-community new-format
Beta(config)# ip bgp-community new-format
Delta(config)# ip bgp-community new-format
```

6.3 Task 13 – Tag AS 180 Customer Routes on Apolo

Apolo tags all routes received from Charlie (AS 180) with community 100:180, identifying them as customer routes. Beta then matches this community and applies higher LOCAL_PREF, ensuring all AS 100 routers prefer Charlie's routes.

Step 1 – Set community on Apolo inbound from Charlie

```
! — Apolo (AS 100) —————
Apolo(config)# route-map TAG-AS180 permit 10
Apolo(config-route-map)# set community 100:180 additive
Apolo(config-route-map)# exit

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.10.20.1 route-map TAG-AS180 in
Apolo(config-router-af)# exit-address-family
! Enable community propagation to Beta via iBGP
```

```

Apolo(config)# router bgp 100
Apolo(config-router)# neighbor 102.16.11.2 send-community both
Apolo# clear ip bgp 102.10.20.1 soft in

Apolo# show bgp ipv4 unicast 192.168.180.0/24
! Should show: Community: 100:180

```

Step 2 – Match community on Beta, set LOCAL_PREF

```

! — Beta (AS 100) —————
Beta(config)# ip community-list standard CUST-180 permit 100:180

Beta(config)# route-map COMMUNITY-POLICY permit 10
Beta(config-route-map)# match community CUST-180
Beta(config-route-map)# set local-preference 200
Beta(config-route-map)# exit
Beta(config)# route-map COMMUNITY-POLICY permit 20
Beta(config-route-map)# exit

Beta(config)# router bgp 100
Beta(config-router)# address-family ipv4 unicast
Beta(config-router-af)# neighbor 102.16.11.1 route-map COMMUNITY-POLICY in
Beta(config-router-af)# exit-address-family
Beta# clear ip bgp 102.16.11.1 soft in

! Verify LP 200 applied on Beta for Charlie's routes
Beta# show bgp ipv4 unicast 192.168.180.0/24
! localpref: 200

```

6.4 Task 14 – no-export Community to Contain Delta Routes

Scenario: AS 100 receives Delta's prefix 172.20.0.0/24 via Beta's eBGP multihop session. This is an AS 147 internal prefix and should not be re-advertised by AS 100 to Charlie (AS 180). Beta applies the well-known no-export community inbound from Delta.

```

! — Beta (AS 100) — tag Delta routes with no-export —————
Beta(config)# ip as-path access-list 50 permit ^147$

Beta(config)# route-map TAG-NO-EXPORT permit 10
Beta(config-route-map)# match as-path 50
Beta(config-route-map)# set community no-export additive
Beta(config-route-map)# exit
Beta(config)# route-map TAG-NO-EXPORT permit 20
Beta(config-route-map)# exit

Beta(config)# router bgp 100
Beta(config-router)# address-family ipv4 unicast
Beta(config-router-af)# neighbor 3.3.3.3 route-map TAG-NO-EXPORT in
Beta(config-router-af)# exit-address-family
Beta(config)# router bgp 100
Beta(config-router)# neighbor 102.16.11.1 send-community both
Beta# clear ip bgp 3.3.3.3 soft in

! Verify: 172.20.0.0/24 must NOT appear on Charlie

```

```
Charlie# show bgp ipv4 unicast | include 172.20
! Expected: no output — route contained within AS 100
```

6.5 Task 15 – Expanded Community-List with Regex

AS 100 uses a structured scheme: 100:1xx = AS 180 customer, 100:2xx = AS 147 peer, 100:3xx = internal. An expanded community-list with regex matches the entire 100:2xx range and sets lower LOCAL_PREF to deprioritise peer routes.

```
! — Beta (AS 100) — tag Delta routes as peer (100:200) —————
Beta(config)# route-map TAG-AS147-PEER permit 10
Beta(config-route-map)# match as-path 50
Beta(config-route-map)# set community 100:200 additive
Beta(config-route-map)# exit

! Expanded community-list — regex matches 100:200 through 100:299
Beta(config)# ip community-list expanded PEER-ROUTES permit 100:2[0-9][0-9]

! Update COMMUNITY-POLICY on Beta to handle peer routes
Beta(config)# route-map COMMUNITY-POLICY permit 10
Beta(config-route-map)# match community CUST-180
Beta(config-route-map)# set local-preference 200
Beta(config-route-map)# exit
Beta(config)# route-map COMMUNITY-POLICY permit 15
Beta(config-route-map)# match community PEER-ROUTES
Beta(config-route-map)# set local-preference 80
Beta(config-route-map)# exit
Beta(config)# route-map COMMUNITY-POLICY permit 20
Beta(config-route-map)# exit

Beta# clear ip bgp 102.16.11.1 soft in
Beta# show bgp ipv4 unicast community 100:200
Beta# show bgp ipv4 unicast community 100:180
```

6.6 Task 16 – Strip Communities Before Advertising Externally

Internal communities (100:xxx) must not leak to external peers. Apply 'set community none' outbound on all eBGP sessions — Apolo toward Charlie, and Beta toward Delta.

```
! — Apolo (AS 100) — strip outbound toward Charlie —————
Apolo(config)# route-map STRIP-COMMUNITIES permit 10
Apolo(config-route-map)# set community none
Apolo(config-route-map)# exit

Apolo(config)# router bgp 100
Apolo(config-router)# address-family ipv4 unicast
Apolo(config-router-af)# neighbor 102.10.20.1 route-map STRIP-COMMUNITIES out
Apolo(config-router-af)# exit-address-family
Apolo# clear ip bgp 102.10.20.1 soft out

! — Beta (AS 100) — strip outbound toward Delta —————
Beta(config)# route-map STRIP-COMMUNITIES permit 10
```

```

Beta(config-route-map)# set community none
Beta(config-route-map)# exit

Beta(config)# router bgp 100
Beta(config-router)# address-family ipv4 unicast
Beta(config-router-af)# neighbor 3.3.3.3 route-map STRIP-COMMUNITIES out
Beta(config-router-af)# exit-address-family
Beta# clear ip bgp 3.3.3.3 soft out

! Verify: no 100:xxx communities visible on Charlie or Delta
Charlie# show bgp ipv4 unicast
Delta# show bgp ipv4 unicast
! Community column should be empty for routes from AS 100

```

7. Verification Command Reference

7.1 Summarization

Command	What to Look For
show bgp ipv4 unicast	'a>' = aggregated best; 's' = suppressed component
show bgp ipv4 unicast X.X.X.X/nn	ATOMIC_AGGREGATE, AGGREGATOR on summary; AS_PATH set
show bgp ipv4 unicast neighbors X advertised-routes	Confirm only summary (not components) sent to peer
show bgp ipv4 unicast neighbors X received-routes	All routes received from peer before inbound filter

7.2 Filtering

Command	What to Look For
show ip access-lists NAME	Hit counts per ACL entry after clear ip bgp soft
show ip prefix-list NAME	Hit counts per prefix-list entry
show ip as-path-access-list N	AS-path ACL entries and hit counts
show bgp ipv4 unicast regexp REGEX	Test regex against live BGP table before deploying
show route-map NAME	Match/set clauses, policy hit counts per sequence

7.3 Communities

Command	What to Look For
show bgp ipv4 unicast community VALUE	Prefixes tagged with a specific community
show bgp ipv4 unicast X.X.X.X/nn	Community field in detailed prefix output

Command	What to Look For
show ip community-list NAME	Community-list entries
show bgp ipv4 unicast neighbors X advertised-routes	Confirm community is/isn't sent to peer

7.4 Common Mistakes & Fixes

Symptom	Likely Cause	Fix
Aggregate not appearing	No component route in BGP table	Inject component via 'network' statement first
Components still advertised	route-map conflict or wrong aggregate command	Check 'summary-only' vs 'suppress-map'; recheck aggregate config
filter-list blocks everything	Missing 'permit .*' catch-all	Add 'ip as-path access-list N permit .*' as last entry
Community not propagating	Missing 'send-community' on neighbor	Add 'neighbor X send-community both'
route-map not applying	Missing 'clear ip bgp X soft'	Always soft-reset after route-map changes
route-map applied at wrong level	Neighbor route-map outside address-family (IOS-XE)	Move 'neighbor X route-map' inside address-family ipv4 unicast
distribute-list blocks too much	Standard ACL ignores mask length	Switch to prefix-list or extended ACL
Regex not matching	Underscore vs space confusion	Use 'show bgp regexp PATTERN' to test before applying

8. Review Questions

Answer all questions based on the confirmed topology: Charlie Gi0/0↔Apolo Gi0/0 (eBGP), Apolo Gi0/1↔Beta Gi0/0 (iBGP), Beta Gi0/1↔Echo Gi0/1 (no BGP), Echo Gi0/0↔Delta Gi0/0 (no BGP), Delta Lo0↔Beta Lo0 (eBGP multihop). Apolo peers with Charlie and Beta only — not with Delta.

8.1 Summarization Questions

Q1. What is the difference between 'aggregate-address X.X.X.X MASK summary-only' and 'aggregate-address X.X.X.X MASK suppress-map RMAP'? When would you choose each?

summary-only suppresses ALL component routes for ALL peers — only the aggregate is advertised to everyone. Use when you want to hide all specifics from all neighbors.

suppress-map suppresses only the components MATCHED by the route-map. Unmatched components continue to be advertised. Use for selective suppression — for example, suppressing /24s toward Charlie (AS180) while still advertising them toward Delta via Beta for traffic engineering.

In this topology: Apolo uses suppress-map to suppress 10.100.0.0/24 while allowing 10.100.1.0/24 (Beta's Lo1) to continue reaching Delta via Beta's eBGP multihop session.

Q2. Charlie advertises 10.180.1-3.0/24 and a summary 10.180.0.0/22 to Apolo. Apolo propagates these to Beta via iBGP. On Beta, 'show bgp ipv4 unicast' shows the /22 aggregate with '>i' but the /24 components are not visible. Explain why.

Charlie has 'summary-only' configured on the aggregate-address 10.180.0.0/22. This suppresses the /24 component routes on Charlie before they are sent — they never leave AS 180.

The '>i' marker on Beta means: '>' = best path, 'i' = learned via iBGP from Apolo. The /24s are suppressed on Charlie and therefore never reach Apolo or Beta.

To verify: 'show bgp ipv4 unicast neighbors 102.10.20.1 received-routes' on Apolo will show only the /22 — confirming the components were suppressed at source.

Q3. Why is 'as-set' important when Apolo creates the 10.100.0.0/22 aggregate from components originating in both Apolo and Beta?

Without as-set, the aggregate has an empty AS_PATH. This is dangerous because: if Charlie (AS180) receives the 10.100.0.0/22 aggregate and re-advertises it, it will have an AS_PATH of just '180'. AS 100 would then accept it back (it has no AS_PATH loop detection signal), potentially creating a routing loop.

With as-set, the aggregate's AS_PATH is set to '{100}' (the set of all component ASes). When AS 100 receives this back from Charlie, it sees its own ASN in the AS_PATH and drops it — standard BGP

loop prevention.

The aggregate will also show `ATOMIC_AGGREGATE` and `AGGREGATOR` attributes, signalling to downstream routers that path information was condensed.

8.2 Filtering Questions

Q4. Write an IOS-XE prefix-list and BGP neighbor command that accepts from Charlie only prefixes in 10.180.0.0/16 with mask /22 to /24, applied inbound on Apollo. Include the full address-family context.

```
ip prefix-list FROM-CHARLIE seq 5 permit 10.180.0.0/16 ge 22 le 24
ip prefix-list FROM-CHARLIE seq 10 deny 0.0.0.0/0 le 32
```

```
router bgp 100
  address-family ipv4 unicast
    neighbor 102.10.20.1 prefix-list FROM-CHARLIE in
```

```
Apollo# clear ip bgp 102.10.20.1 soft in
```

Explanation: 'ge 22 le 24' means the prefix must be within 10.180.0.0/16 AND have a mask length between /22 and /24 inclusive. The seq 10 deny catches everything else.

Q5. What is wrong with applying 'neighbor 102.10.20.1 route-map POLICY in' directly under 'router bgp 100' on IOS-XE? What is the correct location?

On IOS-XE, placing 'neighbor X route-map NAME in|out' directly under 'router bgp' without an address-family context is either rejected or silently ignored. IOS-XE requires all per-neighbor policy commands to be declared within the appropriate address-family block.

Correct location:

```
router bgp 100
  address-family ipv4 unicast
    neighbor 102.10.20.1 route-map POLICY in
```

After changing the configuration, always run 'clear ip bgp 102.10.20.1 soft in' to apply the policy without dropping the session.

8.3 Regex Questions

Q6. Write an AS-path ACL that matches ONLY routes where AS 147 appears more than once in the path (i.e., Delta is doing AS-path prepending). Show how to apply it as a filter-list on Beta inbound from Delta.

```
! Match paths where 147 appears twice or more:
! ^147.* 147 = starts with 147, any content, then 147 again
ip as-path access-list 70 deny ^147.* 147
ip as-path access-list 70 permit .*

router bgp 100
 address-family ipv4 unicast
   neighbor 3.3.3.3 filter-list 70 in

Beta# clear ip bgp 3.3.3.3 soft in

! Test first:
Beta# show bgp ipv4 unicast regexp ^147.* 147
! This blocks '147 147', '147 200 147', etc. but allows '^147$' (single occurrence)
```

Q7. Explain the difference between '^\$', '^147\$', '_147_', and '^147' as BGP AS-path regex patterns. Give an example of what each would match.

```
^$      — Empty AS_PATH. Matches locally originated routes on the router itself (injected via 'network'
or 'redistribute'). Example: Apolo's own 10.100.0.0/24.

^147$   — Exactly the AS_PATH '147' — one ASN, nothing else. Routes originated in AS147 as seen
from a directly-connected eBGP peer. Example: Delta's 172.20.0.0/24 as seen on Beta.

_147_   — AS147 appearing as a whole ASN element anywhere in the path. Matches: '147', '100
147', '100 147 200', '200 147'. Does NOT match '1147' or '1470'. Example: any route that transited or
originated in AS147.

^147    — AS_PATH starting with 147 as the first ASN, followed by anything. Matches: '147', '147
200', '147 200 300'. Does NOT match '100 147'. Example: routes where AS147 is the originating AS
but may have transited further ASes afterward.
```

8.4 Community Questions

Q8. Design a community scheme for AS 100 covering customer (AS180), peer (AS147), and internal routes. Write the community-lists and the route-map on Beta that sets LOCAL_PREF based on community. Include the correct address-family context.

```
Community scheme:
100:180 = AS180 customer routes (LP 200)
100:200 = AS147 peer routes (LP 80)
100:300 = internal AS100 routes (LP 150)
```

```
ip bgp-community new-format
ip community-list standard CUST permit 100:180
ip community-list standard PEER permit 100:200
ip community-list standard INT permit 100:300

route-map APPLY-COMMUNITY-POLICY permit 10
  match community CUST
  set local-preference 200
route-map APPLY-COMMUNITY-POLICY permit 20
  match community INT
  set local-preference 150
route-map APPLY-COMMUNITY-POLICY permit 30
  match community PEER
  set local-preference 80
route-map APPLY-COMMUNITY-POLICY permit 40

router bgp 100
  address-family ipv4 unicast
    neighbor 102.16.11.1 route-map APPLY-COMMUNITY-POLICY in
```

Q9. Beta sets community 100:180 on Charlie's routes inbound and propagates them to Apolo via iBGP. On Apolo, 'show bgp ipv4 unicast 192.168.180.0/24' shows no community. List all possible causes.

Note: In this topology, TAG-AS180 is configured on APOLO inbound from Charlie — not on Beta. If the question assumes Apolo is tagging and Beta is receiving, then the causes are:

1. Missing 'neighbor 102.16.11.2 send-community both' on Apolo — communities are stripped before the iBGP UPDATE reaches Beta.
2. An outbound route-map on Apolo toward Beta uses 'set community none' — stripping the tag before propagation.
3. 'ip bgp-community new-format' not enabled on Apolo or Beta — community shows in decimal rather than AA:NN format; appears to be missing but is actually just displayed differently.
4. The inbound route-map on Apolo from Charlie is applied but the session was not soft-reset — run 'clear ip bgp 102.10.20.1 soft in'.
5. The route-map was applied at the 'router bgp' level instead of inside 'address-family ipv4 unicast' on IOS-XE — silently not applied.

Q10. Challenge — Trace the complete journey of Delta's 172.20.0.0/24 prefix through AS 100: what BGP attributes are set at each hop, where is the community applied, where is it stripped, and what does Charlie ultimately receive?

Step 1 — Delta → Beta (eBGP multihop, Lo0↔Lo0):

UPDATE: NLRI=172.20.0.0/24, NEXT_HOP=3.3.3.3 (Delta Lo0), AS_PATH=147, ORIGIN=IGP.

Beta's inbound route-map TAG-NO-EXPORT matches as-path 50 (^147\$):

→ sets community no-export additive.

Beta's inbound route-map TAG-AS147-PEER also sets community 100:200 additive.

Beta installs: 172.20.0.0/24, next-hop=3.3.3.3 (reachable via OSPFv3 through Echo).

Step 2 — Beta → Apolo (iBGP):

UPDATE: NLRI=172.20.0.0/24, NEXT_HOP=102.16.11.2 (Beta next-hop-self), AS_PATH=147, LP=80 (set by COMMUNITY-POLICY matching 100:200), Community: no-export 100:200.

'neighbor 102.16.11.1 send-community both' must be configured on Beta.

Apolo installs: localpref 80, next-hop 102.16.11.2.

Step 3 — Apolo → Charlie (eBGP):

The no-export community PREVENTS Apolo from advertising this prefix to Charlie.

BGP checks no-export on any prefix before advertising to an eBGP peer and suppresses it automatically — no additional configuration needed.

Charlie receives: NOTHING for 172.20.0.0/24.

This is the intended result — Delta's internal prefix is contained within AS 100.

9. Lab Completion Checklist

Verify each item before signing off. The topology has Apolo peering with Charlie (eBGP) and Beta (iBGP) only — Beta peers with Delta (eBGP multihop).

#	Objective	Status
1	Charlie advertises 10.180.0.0/22 summary to Apolo — /24 components suppressed	☐
2	Apolo advertises 10.100.0.0/22 aggregate with as-set to Charlie — ATOMIC_AGGREGATE confirmed	☐
3	suppress-map allows 10.100.1.0/24 (Beta Lo1) through to Delta despite aggregate	☐
4	distribute-list (std ACL) blocks 10.180.3.0/24 outbound from Apolo to Beta	☐
5	Extended ACL blocks 172.20.x.x more-specifics longer than /24 inbound on Apolo from Beta	☐
6	prefix-list permits only 10.180.0.0/16 ge 22 le 24 inbound on Apolo from Charlie	☐
7	NO-DEFAULT prefix-list applied outbound on Apolo→Charlie and Beta→Delta	☐
8	filter-list blocks routes transiting AS147 on Apolo inbound from Charlie	☐
9	filter-list on Beta accepts only single-ASN-hop routes from Delta (^[0-9]+\$)	☐
10	route-map matches AS180 paths and sets LP=200 on Apolo — inside address-family ipv4	☐
11	Three-clause route-map CHARLIE-INBOUND applies different weights — inside address-family	☐
12	Community 100:180 set on AS180 routes on Apolo; Beta applies LP=200 via community-list	☐
13	no-export community on Delta routes from Beta — 172.20.0.0/24 NOT visible on Charlie	☐
14	Expanded community-list regex 100:2[0-9][0-9] matches 100:200; LP=80 applied on Beta	☐
15	Communities stripped outbound on Apolo→Charlie and Beta→Delta (set community none)	☐
16	All route-map neighbor commands confirmed inside address-family ipv4 unicast on IOS-XE	☐
17	All 10 review questions answered and verified against CLI output	☐

WARN

Save all configurations with 'write memory' on every router. Pay particular attention to Echo — it has no BGP but its OSPF configuration is essential for the Delta↔Beta multihop session.